

Security Audit Report – directus/directus

1. Metadata

Field	Value
Repository	directus/directus
Commit	acfa169
Scan Date	2026-06-05
Report Date	2026-06-05
Auditor	Automated – split model workflow
Status	FINAL – peer review required before action
Classification	Internal – Security Sensitive

2. Executive Summary

Directus is a broadly-deployed open-source headless CMS with a large dependency surface and active release cycle. At commit acfa169, the overall security posture is moderate: the codebase applies sound defensive patterns in its core API (parameterized queries, non-root containers, helmet middleware, role-based access control), but several gaps in supply-chain controls and dependency currency require prompt remediation. The most significant finding is a confirmed path-traversal middleware-bypass vulnerability in the @hono/node-server dependency (GHSA-92pp-h63x-v22m, CVSS 5.3 Medium), which could allow an attacker to access protected static routes by prefixing them with double slashes. The overall Security Posture Score is 6.0 / 10, reflecting solid application-layer defences offset by weak supply-chain controls, an incomplete dependency audit, and absent SBOM/provenance artefacts. The single most important recommended action is upgrading @hono/node-server to 1.19.13 and completing a full npm audit run to quantify the outstanding vulnerability surface.

3. Scope

Area	Artefacts Examined
CI/CD Workflows	20 workflow files under .github/workflows/ including release.yml, codeql-analysis.yml, changeset-check.yml, e2e.yml, claude.yml, cla.yml, blackbox.yml and 13 others
Application Code	api/src/ (operations, services, auth, database helpers), app/src/, packages/, sdk/src/
Dockerfile	Repository root Dockerfile (multi-stage, alpine, node:22)
Dependency Manifests	pnpm-lock.yaml (2,802 packages scanned by OSV-Scanner)
Secrets / Credentials	GitLeaks (aggregate), TruffleHog v3.95.5 (per-file detail)
Container Configuration	Trivy v0.71.0 Dockerfile misconfiguration scan

Area	Artefacts Examined
GitHub API	Branch protection status (404 – not available), Dependabot alerts (403 – access denied), secret scanning status (404 – not available), CODEOWNERS, open issues and PRs
SLSA / Supply Chain	Provenance files, SBOM, Cosign signatures – searched, not present
IaC / Kubernetes	Helm charts, Terraform, Kubernetes manifests – searched, none found
Policy as Code	OPA, Kyverno, Falco – not configured
Static Analysis	Zizmor v1.25.2 (workflow security), actionlint (permission validation); Semgrep not available

4. Methodology

Dimension	Detail
Analysis type	Primarily static; no dynamic testing performed
Dependency scanning	OSV-Scanner against pnpm-lock.yaml; pnpm audit failed due to tool error (reference.startsWith is not a function) – results are incomplete
Secret scanning	TruffleHog v3.95.5 (itemised findings); Gitleaks (aggregate count only – 21 leaks; individual findings not captured)
Workflow analysis	Zizmor v1.25.2 for security anti-patterns; actionlint for permission scopes
Container analysis	Trivy v0.71.0 for Dockerfile misconfigurations
Code review	Pattern-based grep for eval(), Math.random(), raw(), process.exit(), JSON.parse(), fetch(), file operations; no full data-flow tracing tool applied
Known limitations	(1) pnpm audit failed – transitive CVEs beyond OSV-Scanner are unknown. (2) Semgrep not available – SAST coverage is pattern-match only. (3) Gitleaks returned only an aggregate count – 21 findings require manual triage. (4) Branch protection and Dependabot data unavailable via GitHub API. (5) No runtime/DAST testing performed. (6) Data-flow tracing is manual approximation, not tool-verified.

5. Security Strengths

Strength	Evidence
Parameterised SQL throughout	50+ knex.raw() calls consistently use ?? and ? placeholders; no string-concatenation injection patterns detected in data paths
Non-root container execution	Dockerfile runs final image as the node user, not root
Multi-stage Docker build	Builder stage discarded before runtime image; reduces attack surface and image size
Alpine base image	Minimal OS footprint reduces CVE exposure
Helmet middleware active	CSP, HSTS, and CORS configured in api/src/app.ts; X-Powered-By suppressed globally before custom value applied
Role-based access control	sdk/src/schema/permission.ts implements action/collection/field-level rules; authentication middleware enforced in middleware chain
Multiple auth drivers	Cookie, JSON, LDAP, SAML, OAuth2, OpenID â€” broad, modular auth architecture
Rate limiting	Global and IP-based limiters registered in api/src/app.ts:77-78, 312, 316
Startup environment validation	SECRET byte-length check (32-byte minimum), PUBLIC_URL absoluteness, MCP_OAUTH dependency chain, DB connection, migrations, extension, and storage validated before routes serve traffic
Lockfile present	pnpm-lock.yaml ensures reproducible dependency resolution
CodeQL workflow correctly scoped	codeql-analysis.yml uses minimal required permissions (actions: read, contents: read, security-events: write)
Short SARIF artifact retention	CodeQL results retained for 1 day, limiting window of sensitive data exposure
Changeset validation	Structured release notes enforced by CI; private packages correctly excluded
Recent CVE-driven dependency updates	Commit eab59d9 ("CVE dependency updates #27589") demonstrates active security maintenance

6. Findings Summary

ID	Priority	CVSS Severity	Title	OWASP 2021	Status
FIND-001	P1	Medium (5.3)	@hono/node-server Path Traversal Middleware Bypass	A06:2021 â€” Vulnerable and Outdated Components	Open
FIND-002	P2	Medium (4.6)	Unpinned GitHub Actions (write-permission workflows)	A08:2021 â€” Software and Data Integrity Failures	Open
FIND-003	P2	Medium (4.2)	Missing persist-credentials: false on Checkout Steps	A05:2021 â€” Security Misconfiguration	Open
FIND-004	P2	Low (3.3)	@tootallnate/once Indefinite Promise Hang (DoS)	A06:2021 â€” Vulnerable and Outdated Components	Open
FIND-005	P2	Low (3.3)	brace-expansion ReDoS / Memory Exhaustion Bypass	A06:2021 â€” Vulnerable and Outdated Components	Open
FIND-006	P2	Low (2.7)	Invalid models Permission Scope in release.yml	A05:2021 â€” Security Misconfiguration	Open
FIND-007	P2	Medium (5.0)	SSRF Potential in File-Import and Provider Fetch Calls	A10:2021 â€” Server-Side Request Forgery	Potential Attack Surface â€” Requires Confirmation
FIND-008	P3	Low (2.0)	Missing Dockerfile HEALTHCHECK	A05:2021 â€” Security Misconfiguration	Open
FIND-009	P3	Informational	Weak Hash Functions in Non-Security Contexts	A02:2021 â€” Cryptographic Failures	Open
FIND-010	P3	Low (2.3)	X-Powered-By Header Re-enabled with Server Identity	A05:2021 â€” Security Misconfiguration	Open
[Gitleaks]	P2	Unrated â€” pending triage	Gitleaks: 21 aggregate findings â€” manual triage required	A02:2021 â€” Cryptographic Failures	Pending triage

7. Security Posture Summary

Area	Score (0â€”5)	Rationale
CI/CD Pipeline Security	5.5	Workflows are structurally sound (scoped permissions in CodeQL, changeset logic correct); offset by unpinned actions across 20 workflows, missing credential suppression, and an invalid permission scope in the release pipeline
Dependency Management	5.0	Lockfile present; OSV-Scanner confirms three exploitable CVEs; pnpm audit failed â€” transitive vulnerability surface unknown; one Medium CVE unpatched
Secrets Management	7.0	No hardcoded production secrets found; TruffleHog 8 findings are all test fixtures; Gitleaks 21-finding aggregate untriaged; branch protection / secret-scanning enablement unconfirmed
Supply Chain Integrity	3.5	No SBOM, no provenance, no Cosign signatures, no SLSA workflow; all actions use mutable version tags; active CVE maintenance present but supply chain attestation absent
Container Security	7.5	Non-root user, multi-stage build, Alpine base, lockfile â€” strong posture; only missing HEALTHCHECK instruction

Area	Score (0-10)	Rationale
Application Code (SAST)	6.5	Parameterised SQL throughout; helmet/rate-limiting/RBAC active; eval sandboxed; SSRF surface unconfirmed; weak hashes non-security only; Semgrep unavailable
Overall (weighted average)	6.0	Weighted: CI/CD –20%, Deps –20%, Secrets –15%, Supply Chain –15%, Container –15%, App Code –15%

8. Per-Finding Sections

FIND-001 – @hono/node-server Path Traversal Middleware Bypass

Field	Value
Priority	P1
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CVSS Score	5.3 Medium
OWASP 2021	A06 – Vulnerable and Outdated Components
CWE	CWE-22 – Improper Limitation of a Pathname to a Restricted Directory
Affected Package	@hono/node-server@1.19.10 (in pnpm-lock.yaml)
Advisory	GHSA-92pp-h63x-v22m / CVE-2026-39406
Fixed Version	1.19.13

Description

The `serveStatic` middleware in `@hono/node-server` versions prior to 1.19.13 fails to normalise repeated leading slashes in request paths. A request such as `GET //admin/secret.txt` bypasses middleware guards matching `/admin/*`, allowing unauthenticated access to protected static assets.

Root Cause

The middleware performs route matching on the raw path string without collapsing duplicate leading slashes before pattern evaluation. The route guard `/admin/*` matches `/admin/...` but not `//admin/...`, creating a bypass.

Impact Chain

1. Attacker sends `GET //admin/config.json` or equivalent double-slash prefixed path.
2. Route-level access guard (`/admin/*`) does not match.
3. `serveStatic` resolves and serves the underlying file from disk.
4. Attacker reads protected configuration or sensitive static files served by this middleware.
5. Depending on what static files are served, this could lead to credential or configuration disclosure.

Fix

Update `@hono/node-server` to `>= 1.19.13` in the relevant `package.json`:

```
// In the package.json of the package consuming @hono/node-server
{
  "dependencies": {
    "@hono/node-server": "^1.19.13"
  }
}
```

If the dependency is transitive, add a pnpm override at the workspace root:

```
// Root package.json
{
  "pnpm": {
    "overrides": {
      "@hono/node-server": ">=1.19.13"
    }
  }
}
```

Then run:

```
pnpm install
```

Verification

```
# Confirm installed version after update
pnpm list @hono/node-server --depth 0

# Re-run OSV-Scanner to confirm advisory cleared
osv-scanner --lockfile pnpm-lock.yaml | grep -i hono

# Functional test: confirm double-slash path is blocked (adjust host/path as appropriate)
curl -sv "http://localhost:8055//admin/secret.txt" 2>&1 | grep -E "HTTP/|< HTTP"
```

FIND-002 – Unpinned GitHub Actions in Write-Permission Workflows

Field	Value
Priority	P2
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:C/C:L/I:L/A:N
CVSS Score	4.6 Medium
OWASP 2021	A08 – “Software and Data Integrity Failures”
CWE	CWE-829 – “Inclusion of Functionality from Untrusted Control Sphere”

Field	Value
Affected Workflows	release.yml (uses docker/build-push-action@v6, sigstore/cosign-installer@v3, directus/npm-package-existence-checker@v1); cla.yml (uses marocchino/sticky-pull-request-comment@v2); claude.yml and claude-code-review.yml (uses anthropics/claude-code-action@v1)

Description

All 20 CI/CD workflows pin GitHub Actions to mutable major-version tags (e.g., @v6, @v4, @v1) rather than immutable commit SHAs. For several of these workflows – particularly release.yml, which publishes Docker images and npm packages – the workflow grants elevated permissions (packages: write, contents: write implied by release operations) and is triggered by authenticated pushes. If a third-party action publisher is compromised and a malicious commit is pushed under the same tag, the compromised code executes in the context of these elevated permissions.

The risk is rated Medium (not High) because: (a) attacker must first compromise the upstream action repository, (b) the release.yml trigger is push to protected branches (not pull_request_target or issue_comment), and (c) claude.yml and claude-code-review.yml handle AI interaction with access to pull-request context but the exact permission scope is not fully captured.

The anthropics/claude-code-action@v1 and directus/npm-package-existence-checker@v1 actions are first- or closely-affiliated party actions with no documented vetting cadence in the evidence.

Root Cause

No repository-level policy enforces SHA pinning for Actions. Developers default to convenience version tags. The OpenSSF Scorecard "Pinned-Dependencies" check would flag this automatically if enabled.

Impact Chain

1. Attacker compromises a third-party action's repository (e.g., docker/build-push-action) and pushes malicious code under the @v6 tag.
2. On next push trigger to the main branch, release.yml fetches the now-malicious action.
3. Action executes arbitrary code in the runner with access to GITHUB_TOKEN (which has write permissions in the release context) and any repository secrets.
4. Attacker exfiltrates secrets, inserts malicious code into published Docker image or npm package, or modifies release artefacts.

Fix

Pin each action to a full commit SHA. Example for release.yml:

```
# Before
- uses: docker/build-push-action@v6

# After – replace SHA with the resolved value from the action's release tag
# Run: git ls-remote https://github.com/docker/build-push-action refs/tags/v6
- uses: docker/build-push-action@<commit-SHA-here>
  # v6 – keep tag as a comment for human readability
```

Use a tool such as pin-github-action or Dependabot (with versioning-strategy: lockfile-only) to automate SHA resolution and keep SHAs current:

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

Note on Resolved Pin SHAs: The collected evidence does not include resolved commit SHAs for any of the affected actions. Do not substitute SHAs until resolved from the authoritative action repositories using the `git ls-remote` command above.

Verification

```
# Find all unpinned action references across all workflows
grep -rn 'uses:' .github/workflows/ | grep -v '@[0-9a-f]\{40\}'

# After pinning, confirm no mutable tags remain
grep -rn 'uses:' .github/workflows/ | grep '@v[0-9]'
# Expected: no output
```

FIND-003 – Missing persist-credentials: false on Checkout Steps

Field	Value
Priority	P2
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:C/C:L/I:L/A:N
CVSS Score	4.2 Medium
OWASP 2021	A05 – Security Misconfiguration
CWE	CWE-522 – Insufficiently Protected Credentials
Confirmed Affected	.github/workflows/agent-scan.yml
Likely Affected	All workflows using actions/checkout without explicit credential suppression

Description

When actions/checkout runs without `persist-credentials: false`, the `GITHUB_TOKEN` credential is written to the local Git configuration (`~/.config/git/credentials`) for the duration of the workflow. Any subsequent step – including scripts executed by third-party actions – can read this credential and use it to authenticate to the GitHub API with the permissions of the workflow's `GITHUB_TOKEN`.

Zizmor v1.25.2 confirmed this finding in `.github/workflows/agent-scan.yml`. The full scope across all 20 workflows is not fully captured (Zizmor output was truncated).

Root Cause

The actions/checkout action persists credentials by default to support follow-on Git operations (push, fetch). When subsequent Git operations are not required, this default creates an unnecessary credential exposure window.

Impact Chain

1. A compromised or malicious step in a workflow (e.g., a compromised third-party action as per FIND-002) executes after actions/checkout.

2. Step reads the persisted GITHUB_TOKEN from local Git config.
3. Token is used to authenticate to the GitHub API within its permission scope.
4. Depending on workflow permissions, attacker may read repository contents, write comments, push branches, or access package registries.

Fix

Add persist-credentials: false to all actions/checkout steps where subsequent authenticated Git operations are not required:

```
- name: Checkout repository
  uses: actions/checkout@v4 # (pin to SHA per FIND-002)
  with:
    persist-credentials: false
```

If a subsequent step requires Git push/fetch, retain credentials only for that specific workflow and document the reason in a comment.

Verification

```
# Find all checkout steps lacking persist-credentials: false
grep -n 'actions/checkout' .github/workflows/*.yaml

# For each workflow, check whether persist-credentials: false is present
grep -A5 'actions/checkout' .github/workflows/agent-scan.yaml | grep 'persist-credentials'

# After fix, confirm all checkout steps have the flag
grep -B2 -A10 'actions/checkout' .github/workflows/*.yaml | grep -c 'persist-credentials: false'
```

FIND-004 – @tootallnate/once Indefinite Promise Hang (DoS)

Field	Value
Priority	P2
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L
CVSS Score	3.3 Low
OWASP 2021	A06 – Vulnerable and Outdated Components
CWE	CWE-400 – Uncontrolled Resource Consumption
Affected Package	@tootallnate/once@1.1.2 and @tootallnate/once@2.0.0 (in pnpm-lock.yaml)
Advisory	GHSA-vpq2-c234-7xj6 / CVE-2026-3449
Fixed Version	v2.0.1 and v3.0.1

Description

Incorrect control-flow handling when an AbortSignal is triggered causes promises to hang indefinitely, stalling associated request handlers or worker threads. This could degrade server responsiveness under conditions where requests are aborted.

Root Cause

The library fails to resolve or reject the promise when the abort signal fires, leaving the promise in a permanently pending state.

Impact Chain

1. A client or internal component initiates a request with an AbortSignal.
2. The signal fires (e.g., timeout, client disconnect).
3. The promise created by @tootallnate/once never settles.
4. Worker thread or event-loop slot remains occupied indefinitely.
5. Under sustained conditions, available worker slots are exhausted, causing service degradation.

Fix

```
// Root package.json â€” add pnpm override
{
  "pnpm": {
    "overrides": {
      "@tootallnate/once": ">=2.0.1"
    }
  }
}
```

```
pnpm install
pnpm list @tootallnate/once
```

Verification

```
# Confirm installed version after override
pnpm list "@tootallnate/once" --depth 10

# Re-run OSV-Scanner
osv-scanner --lockfile pnpm-lock.yaml | grep -i tootallnate
# Expected: no findings
```

FIND-005 â€” brace-expansion Memory Exhaustion DoS Bypass

Field	Value
Priority	P2
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L
CVSS Score	3.3 Low
OWASP 2021	A06 â€” Vulnerable and Outdated Components
CWE	CWE-770 â€” Allocation of Resources Without Limits or Throttling

Affected Package	brace-expansion@5.0.5 (in pnpm-lock.yaml)
Advisory	GHSA-jxxr-4gwj-5jf2 / CVE-2026-45149
Fixed Version	5.0.6

Description

The max option intended to limit brace expansion output is applied after the expansion is computed, not before. An input such as {1..10000000} allocates approximately 505 MB of memory before the limit is checked. If user-controlled input reaches this library, a single request can cause significant memory pressure.

Root Cause

Evaluation order bug: the expansion algorithm completes full allocation before consulting the configured limit, defeating the protective intent of the max option.

Impact Chain

1. Attacker sends a request containing a large brace-expansion expression to an endpoint that passes the value to brace-expansion.
2. Library allocates ~505 MB per request before the limit is enforced.
3. Under concurrent requests, heap memory exhaustion causes Node.js process crash or OOM kill.
4. Service becomes unavailable (Denial of Service).

Note: Exploitability depends on whether user-controlled input reaches brace-expansion. This is a transitive dependency; the exact code path in Directus is not confirmed from static evidence alone.

Fix

```
// Root package.json
{
  "pnpm": {
    "overrides": {
      "brace-expansion": ">=5.0.6"
    }
  }
}
```

```
pnpm install
```

Verification

```
pnpm list brace-expansion --depth 10
```

```
osv-scanner --lockfile pnpm-lock.yaml | grep -i brace-expansion
# Expected: no findings
```

FIND-006 Invalid models Permission Scope in release.yml

Field	Value
Priority	P2
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:H/PR:H/UI:N/S:U/C:N/I:L/A:N
CVSS Score	2.7 Low
OWASP 2021	A05 "Security Misconfiguration"
CWE	CWE-732 "Incorrect Permission Assignment for Critical Resource"
Affected File	.github/workflows/release.yml lines 57, 102, 209
Tool	actionlint

Description

Three job-level permissions blocks in release.yml declare models: read or equivalent. The models scope is not a valid GitHub Actions permission scope. GitHub ignores unknown scopes silently; the permission block does not fail. The intended permission is unclear "if models was intended to grant access to a specific

GitHub resource (e.g., for an AI provider integration), that access is not being granted, creating an unintended behaviour gap. Additionally, the presence of unknown scopes may indicate a broader copy-paste misconfiguration that could inadvertently grant more permissions than intended elsewhere.

Root Cause

Likely copy-paste from documentation or configuration for an AI/ML integration (e.g., GitHub Models) that does not yet correspond to a stable, documented GitHub Actions permission scope. The invalid scope was not caught before merge because GitHub silently ignores unknown permission keys.

Impact Chain

1. Workflow executes with models scope declared but not applied.
2. If the intended capability required the (invalid) scope, it silently fails to function.
3. Alternatively, if a future GitHub API version recognises models as a valid scope with elevated privileges, existing workflow configs may unintentionally grant those privileges without a deliberate policy decision.

Fix

Remove or replace the invalid scope. If the intent is to permit GitHub Models API access, verify the current documented scope name:

```
# Before (release.yml, line ~57)
permissions:
  contents: write
  packages: write
  models: read      # â†• remove or replace with correct scope

# After
permissions:
  contents: write
  packages: write
  # models scope is not valid; remove until GitHub documents it officially
```

Verification

```
# Confirm the scope is present
grep -n 'models' .github/workflows/release.yml

# After fix, run actionlint
actionlint .github/workflows/release.yml
# Expected: no "unknown permission scope" errors
```

FIND-007 • SSRF Potential in File-Import and Provider Fetch Calls

Field	Value
Priority	P2
CVSS v3.1	CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N
Vector	
CVSS Score	5.0 Medium

Field	Value
OWASP 2021	A10 " Server-Side Request Forgery
CWE	CWE-918 " Server-Side Request Forgery
Status	Potential Attack Surface " Requires Confirmation
Affected Files	api/src/services/files.ts:285, api/src/ai/providers/ (multiple), api/src/permissions/utis/fetch-dynamic-variable-data.ts

Description

Multiple `fetch()` calls are made to URLs derived from configuration or user-supplied input. Specifically, `files.ts:285` performs an HTTP request to import files from a URL, AI provider files in `api/src/ai/providers/` call external APIs, and `fetch-dynamic-variable-data.ts` fetches data for dynamic permission variables. No URL allowlisting or scheme validation was observed in the static evidence for these paths.

This is classified as a Potential Attack Surface because data-flow tracing from the HTTP request parameter to the `fetch()` call has not been confirmed through dynamic analysis or full AST-level data-flow tracing.

Root Cause

File-import-by-URL is a documented feature of Directus. If the provided URL is not validated against an allowlist of schemes (must be `https://`) and hosts (or blocked ranges), an authenticated user could cause the server to issue requests to internal network addresses (169.254.x.x, 10.x.x.x, localhost), metadata services, or other internal endpoints.

Impact Chain (if URL validation is absent)

1. Authenticated user with file-upload permission submits a URL pointing to `http://169.254.169.254/latest/meta-data/` (AWS metadata endpoint) or an internal service.
2. Directus server issues the HTTP request from its network context.
3. Response content is returned to or stored by the application.
4. Attacker reads cloud instance credentials, internal API responses, or other network-internal resources.

Fix

Implement URL validation before any outbound fetch:

```
// Recommended: validate URL before fetch in files.ts and similar locations
import { URL } from 'url';

function isSafeURL(rawUrl: string): boolean {
  try {
    const parsed = new URL(rawUrl);
    if (!['https:'].includes(parsed.protocol)) return false;
    // Block private/loopback ranges " use a library such as `is-ip` + CIDR check
    const blocked = ['localhost', '127.0.0.1', '0.0.0.0', '169.254.169.254'];
    if (blocked.some(h => parsed.hostname === h)) return false;
    return true;
  } catch {
    return false;
  }
}
```

Additionally, consider adding `IMPORT_IP_DENY_LIST` or equivalent environment-variable-driven blocklist

(a pattern already seen in f75b25f commit "IP blocklist update").

Verification

```
# Locate all fetch() calls in API source to enumerate attack surface
grep -rn 'fetch(' api/src/ | grep -v '.test.ts' | grep -v 'node_modules'
```

```
# After fix: confirm URL validation is called before each external fetch
grep -n 'isSafeURL\|isDenied\|IMPORT_IP_DENY' api/src/services/files.ts
```

```
# Dynamic
```

Appendix: Claim Verification

Automated post-generation check of concrete claims against collected evidence. Method: SHAs matched against resolved evidence.

Claim	Type	Status	Detail
--- --- --- ---			
`CVE-2026-39406`	CVE	â€œ verified	found in Dependabot/dependency evidence
`CVE-2026-3449`	CVE	â€œ verified	found in Dependabot/dependency evidence
`CVE-2026-45149`	CVE	â€œ verified	found in Dependabot/dependency evidence
`5.3 (Medium)`	CVSS band	â€œ verified	band correct
`4.6 (Medium)`	CVSS band	â€œ verified	band correct
`4.2 (Medium)`	CVSS band	â€œ verified	band correct
`3.3 (Low)`	CVSS band	â€œ verified	band correct
`2.7 (Low)`	CVSS band	â€œ verified	band correct
`5.0 (Medium)`	CVSS band	â€œ verified	band correct
`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`	CVSS vector	â€œ verified	computed 5.3 (Medium)
`CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:C/C:L/I:L/A:N`	CVSS vector	â€œ verified	computed 4.7 (Medium)
`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:C/C:L/I:L/A:N`	CVSS vector	â€œ verified	computed 4.9 (Medium)
`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L`	CVSS vector	â€œ verified	computed 3.7 (Low)
`CVSS:3.1/AV:N/AC:H/PR:H/UI:N/S:U/C:N/I:L/A:N`	CVSS vector	â€œ verified	computed 2.2 (Low)
`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N`	CVSS vector	â€œ verified	computed 4.2 (Medium)
`#27589`	PR/issue	â€œ verified	present in GitHub/commit evidence
`api/src/app.ts:77`	file:line	â€œ verified	found in grep evidence
`api/src/services/files.ts:285`	file:line	â€œ verified	found in grep evidence
`files.ts:285`	file:line	â€œ verified	found in grep evidence
`node-server@1.19.10`	pkg@version	â€œ verified	package name found in dependency evidence (version not matched exact)
`once@1.1.2`	pkg@version	â€œ verified	package name found in dependency evidence (version not matched exact)
`once@2.0.0`	pkg@version	â€œ verified	package name found in dependency evidence (version not matched exact)
`brace-expansion@5.0.5`	pkg@version	â€œ verified	found in dependency evidence
`.github/workflows/agent-scan.yml`	workflow file	â€œ verified	matches collected workflow list
`.github/workflows/release.yml`	workflow file	â€œ verified	matches collected workflow list

****Summary:**** 25 verified, 0 unverified.